

ServeNow

ServeNow

Complete Home Services Platform

Developer Documentation & Buyer Guide

Version 2.0 · March 2026

Flutter

Firebase

Razorpay

Google Maps

Customer App · Provider App · Handyman App · Flutter Web Admin Panel

Flutter + Firebase · No PHP Server · No Hosting Costs for Admin Panel

Table of Contents

1. Platform Overview

- 1.1 What is ServeNow?
- 1.2 Tech Stack

2. Feature List

- 2.1 Customer App
- 2.2 Provider App
- 2.3 Handyman App
- 2.4 Admin Web Panel

3. Project Structure

- 3.1 Folder Structure
- 3.2 Firebase Collections

4. Setup Guide

- 4.1 Prerequisites
- 4.2 Firebase Project Setup
- 4.3 Flutter Apps Configuration
- 4.4 Admin Web Panel Setup
- 4.5 Cloud Functions Deployment
- 4.6 API Keys Configuration

5. Rebranding Guide

- 5.1 Rename the App
- 5.2 Change Package Names
- 5.3 Update Colors & Theme
- 5.4 Replace Icons & Logos

6. Going Live Checklist

7. Support & License

1. Platform Overview

1.1 What is ServeNow?

ServeNow is a complete, production-ready home services marketplace platform built entirely with Flutter and Firebase. It connects customers who need home services (plumbing, electrical, cleaning, carpentry, etc.) with service providers and their field handymen — all managed through a powerful Flutter Web admin panel hosted on Firebase Hosting.

Component	Technology	Purpose
Customer App	Flutter (Android + iOS)	Browse, book, track and pay for services
Provider App	Flutter (Android + iOS)	Manage bookings and earnings
Handyman App	Flutter (Android + iOS)	Accept jobs, navigate and collect payments
Admin Web Panel	Flutter Web + Firebase Hosting	Full platform control — no server needed
Database	Firebase Firestore (asia-south1)	Real-time NoSQL database
Auth	Firebase Authentication	Phone OTP + Google Sign-In
Storage	Firebase Storage	Profile photos, ID proofs, documents
Functions	Firebase Cloud Functions (asia-south1)	FCM notifications, auto-assignment, commissions
Payments	Razorpay	Online payments, wallet top-up (buyer configures own keys)
Maps	Google Maps SDK	Live tracking, navigation, geocoding

1.2 Tech Stack

Technology	Version	Usage
Flutter / Dart	3.x	All 4 apps including admin web — clean architecture, Riverpod/GetX
Firebase Firestore	Latest SDK	Real-time database — asia-south1 region
Firebase Auth	Latest SDK	Phone OTP + Google Sign-In across all apps
Firebase Storage	Latest SDK	File uploads — photos, documents
Firebase Functions	Node.js 18	Auto-assignment, FCM, commission logic — asia-south1
Firebase Hosting	Latest	Admin web panel hosted free — no server required
Razorpay Flutter SDK	1.3.6	Payment gateway — buyer must configure own API keys
Google Maps Flutter	2.5	Live handyman tracking, navigation, geocoding
GetX	4.6.6	State management + routing in admin_web
Riverpod	2.x	State management in customer, provider, handyman apps

2. Feature List

2.1 Customer App

Feature	Description
Phone + OTP Auth	Firebase phone authentication with 6-digit OTP and 30-second resend timer
Google Sign-In	One-tap login — auto-fills name and profile photo from Google account
Profile Management	Edit name, photo, email. Manage multiple saved addresses with labels
Service Browsing	Browse services by category with real-time pricing from Firestore
Provider Selection	Choose from approved providers or book any available provider (broadcast)
Booking Flow	Select service, provider, date/time, address. COD or online payment
Online Payment	Razorpay integration — UPI, cards, netbanking (buyer configures own keys)
Cash on Delivery	Book now and pay the handyman on arrival
Coupon Support	Apply coupon codes with validation (type, expiry, min order, usage limit)
Real-time Status	Visual step timeline: Pending → Confirmed → Assigned → En Route → Arrived → In Progress → Completed
OTP Sharing	Auto-generated 4-digit OTP shown when handyman is assigned — shared to start job
Live Tracking	Google Maps with animated real-time handyman marker and route polyline
Ratings & Reviews	Rate provider and handyman post-job with star rating and review text
Push Notifications	FCM at every booking status change with deep link to booking detail
Dark Mode	Full dark mode with persistent preference

2.2 Provider App

Feature	Description
5-Step Onboarding	Business info, service categories, service areas, ID proof upload, bank details
Admin Approval Flow	Pending approval screen with real-time listener — auto-navigates when approved
Real-time Dashboard	Live booking stats, pending request banner, quick action buttons
Accept and Decline	One-tap accept or decline on each new pending booking request
Broadcast Bookings	Receives new booking requests for their service categories automatically
Auto Handyman Assignment	System auto-assigns nearest online handyman 3 minutes after booking confirmed
Booking Tabs	New Requests / Active / Completed / Cancelled with live badge counts
Earnings and Wallet	Balance card, total earned, transaction history with providerEarning per booking
Dark Mode	Full dark mode with persistent preference
Push Notifications	FCM for new bookings, cancellations, approvals and withdrawal updates

2.3 Handyman App

Feature	Description
Onboarding & Approval	Registration with ID proof upload — admin must approve before accessing the app
Online/Offline Toggle	Prominent toggle on dashboard — GPS tracking starts when toggled online
Auto-Assignment	Jobs assigned automatically by Cloud Function based on proximity and availability
Job List with Tabs	Assigned / Active / Completed / Cancelled with status-colored left borders
Status Progression	Start Journey → Arrived → Enter OTP → Complete Job
OTP Verification	4-digit OTP entry screen — must match customer OTP to start job
Google Maps Navigation	Opens Google Maps with customer coordinates for turn-by-turn directions
COD Cash Collection	Mandatory collection dialog showing exact amount before completing COD jobs
Wallet	Balance, earnings history, cash collected log. Wallet top-up via Razorpay (requires setup)
Commission Rate	Configurable per handyman — admin sets percentage, handyman can view in profile
Push Notifications	FCM for job assignments, status changes, chat messages
Green Theme	Distinct forest green identity separate from other apps

2.4 Admin Web Panel

The admin panel is built in Flutter Web and hosted on Firebase Hosting — no PHP server, no Laravel, no hosting costs. Access it from any browser at your Firebase Hosting URL.

Section	Features
Dashboard	Live counters: bookings, revenue, pending, active. Charts for revenue and status
Provider Approvals	Real-time badge count. Full detail card with ID proof, approve/reject with FCM notification
Handyman Approvals	Review handyman applications with ID proof, approve/reject, set commission rate
Bookings	Real-time stream, status filter, override status, reassign, amount override, admin notes
Customers	View, block/unblock, soft delete, booking history
Providers	View approved providers, commission override, activate/deactivate, wallet history
Handymen	View all handymen, activate/deactivate, job history, commission rate management
Service Categories	Add/edit/reorder categories. Controls customer app browsing
Service Pricing	Manage services and prices grouped by category — reflects instantly in customer app
Coupons	Full coupon management: percent or flat discount, usage limits, validity dates
Withdrawals	Approve/reject payout requests with wallet adjustment and FCM
Announcements	Send targeted FCM push to customers, providers, handymen or all users
Dark Mode	Full dark mode toggle with localStorage persistence
Hosting	Deployed to Firebase Hosting — free, fast, no server required

3. Project Structure

3.1 Folder Structure

All five components live under a single project root:

```

ServeNow/
├── customer_app/           Flutter app for customers (Android + iOS)
│   ├── lib/
│   │   ├── core/          Theme, router, services, shared widgets
│   │   └── features/       auth, booking, tracking, chat, ratings, profile
│   └── provider_app/      Flutter app for service providers (Android + iOS)
│       ├── lib/
│       │   ├── core/
│       │   └── features/   auth, onboarding, dashboard, bookings, earnings
│       └── handyman_app/  Flutter app for field handymen (Android + iOS)
│           ├── lib/
│           │   ├── core/
│           │   └── features/ auth, onboarding, jobs, wallet, profile
│           └── admin_web/  Flutter Web admin panel (Firebase Hosting)
│               ├── lib/
│               │   ├── app/    models, routes, utils, modules (GetX pattern)
│               │   └── widgets/ menu, layout, stat cards, data tables
│               └── functions/  Firebase Cloud Functions (Node.js 18)
│                   ├── index.js All triggers: booking lifecycle, auto-assignment,
│                   │           FCM notifications, commission crediting
│                   └── package.json
│               └── scripts/    Utility scripts
│                   └── seed_all.js Demo data seeding script for buyers

```

3.2 Firebase Collections

Collection Path	Description	Key Fields
customers/{uid}	Customer profiles	fullName, phone, isProfileComplete, isBlocked, fcmToken
customers/{uid}/addresses	Saved addresses	label, fullAddress, area, lat, lng, isDefault
providers/{uid}	Provider profiles	businessName, onboardingStatus, commissionRate, rating, categories
handymen/{uid}	Handyman profiles	name, phone, isApproved, onboardingStatus, commissionRate, isOnline, skills
bookings/{id}	All booking records	status, otp, amount, commission, providerEarning, handymanEarning, assignedHandymanId
categories/{id}	Service categories	name, iconName, color, isActive, sortOrder
services/{id}	Services and pricing	name, basePrice, categoryId, categoryName, duration, isActive, bookingCount

Collection Path	Description	Key Fields
coupons/{code}	Discount coupons	type, discount, minOrder, usageLimit, usageCount, isActive, expiresAt
banners/{id}	Home screen banners	title, subtitle, imageUrl, actionType, isActive, sortOrder
ratings/{id}	Customer reviews	providerRating, handymanRating, review, isVisible, createdAt
app_config/settings	Global platform config	commissionRate, currency, supportEmail, supportPhone
admins/{uid}	Admin user accounts	role (superadmin/manager/support), isActive, email, name

4. Setup Guide

4.1 Prerequisites

Tool	Version	Notes
Flutter SDK	3.x or higher	https://flutter.dev — required for all 4 apps including admin web
Node.js	18.x or higher	Required for Firebase Cloud Functions
Firebase CLI	Latest	<code>npm install -g firebase-tools</code>
FlutterFire CLI	Latest	<code>dart pub global activate flutterfire_cli</code>
Android Studio	Latest	Or VS Code with Flutter and Dart extensions
Google Account	-	For Firebase Console and Google Cloud Console
Razorpay Account	-	Sign up at razorpay.com — free sandbox available

4.2 Firebase Project Setup

- Step 1** Go to console.firebase.google.com and create a new project. Name it after your brand.
- Step 2** Enable Authentication. Go to Sign-in methods and enable both Phone and Google providers.
- Step 3** Create Firestore Database. Select asia-south1 region. Start in test mode initially.
- Step 4** Enable Firebase Storage in test mode for photos and documents.
- Step 5** Cloud Messaging (FCM) is enabled automatically — no extra setup needed.
- Step 6** Enable Firebase Hosting — this is where the admin web panel will be deployed for free.
- Step 7** Add Android app for `customer_app` with package name `com.serveNOW.customer`. Download `google-services.json` and place in `customer_app/android/app/`
- Step 8** Repeat Step 7 for `provider_app` (`com.serveNOW.provider`) and `handyman_app` (`com.serveNOW.handyman`).
- Step 9** Add a Web app to Firebase project. Download the firebase config — needed for `admin_web` flutterfire configure.
- Step 10** Add iOS apps and place each `GoogleService-Info.plist` inside `ios/Runner/` of the respective app.

4.3 Flutter Apps Configuration

Repeat for `customer_app`, `provider_app`, and `handyman_app`:

- Replace `google-services.json` in `android/app/` with the file from your Firebase project.
- Replace `GoogleService-Info.plist` in `ios/Runner/` with the file from Firebase.
- Open `android/app/build.gradle` — verify `applicationId` matches your Firebase app.
- Run: `flutter pub get`
- Run: `dart run build_runner build --delete-conflicting-outputs`
- Run: flutter analyze and fix any errors.
- Run on device: `flutter run`

4.4 Admin Web Panel Setup

The admin panel is a Flutter Web app — no PHP, no server, no Laravel. It runs entirely in the browser and connects directly to Firestore.


```
cd path/to/ServeNow/admin_web

# Configure Firebase for web
flutterfire configure --project=your-firebase-project-id
# Select: Web platform only – generates lib/firebase_options.dart

flutter pub get

# Run locally in Chrome for testing
flutter run -d chrome

# Build for production
flutter build web --release --web-renderer canvaskit

# Deploy to Firebase Hosting (free)
firebase deploy --only hosting

# Admin panel live at: https://your-project-id.web.app
```

Create your first superadmin by adding a document to `admins/{uid}` in Firestore with fields: `name`, `email`, `role: "superadmin"`, `isActive: true`. Use the Firebase Auth UID as the document ID.

4.5 Cloud Functions Deployment

```
firebase login
firebase use --add # select your Firebase project

cd functions && npm install

# Verify region in index.js – must be asia-south1
firebase deploy --only functions

# Verify in Firebase Console → Functions
# All functions should show region: asia-south1
```

■ **Cloud Functions MUST be deployed to asia-south1 to match your Firestore database region. If functions are in us-central1, booking triggers (auto-assignment, notifications, commissions) will NOT fire.**

4.6 API Keys Configuration

API Key	Where to Get It	Where to Place It
Google Maps API Key	console.cloud.google.com → APIs and Services → Credentials	AndroidManifest.xml of customer_app and handyman_app. Also AppDelegate.swift for iOS.
Razorpay Key ID	dashboard.razorpay.com → Settings → API Keys → Generate Test Key	Search rzp_test_ placeholder in customer_app and handyman_app — replace with your key ID.
Razorpay Key Secret	Same Razorpay dashboard page	Used server-side in functions/index.js only. NEVER place in Flutter app code.
Firebase Config	Firebase Console → Project Settings → Your apps → Web app	Run flutterfire configure in admin_web — auto-generates firebase_options.dart

■ **Enable these Google Cloud APIs: Maps SDK for Android, Maps SDK for iOS, Directions API, Distance Matrix API, Geocoding API. All five are required for tracking and address features.**

Razorpay Setup Steps

- Sign up at razorpay.com (free account, no credit card needed for sandbox)
- Go to Settings → API Keys → Generate Test Key
- Copy your Key ID (starts with rzp_test_) and Key Secret
- In functions/index.js: set key_id and key_secret in the Razorpay initialization
- In customer_app/lib/core/services/razorpay_service.dart: replace the _keyId placeholder with your Key ID
- Deploy Cloud Functions: firebase deploy --only functions:createRazorpayOrder
- For production: generate Live Keys (rzp_live_) from the same dashboard

Seeding Demo Data

A demo data script is included to populate your Firebase with realistic test data:

```
cd scripts
npm install
# Place firebase-service-account.json in the scripts/ folder
# (Download from Firebase Console → Project Settings → Service Accounts)
node seed_all.js

# This seeds:
#   categories/      - 5 service categories
#   services/        - 15 services (3 per category)
#   banners/         - 3 home screen banners
#   coupons/         - 4 discount coupon codes
#   app_config/      - default platform settings
```

5. Rebranding Guide

5.1 Rename the App

What to Change	File Location	Value to Update
Android name (Customer)	customer_app/android/app/src/main/AndroidManifest.xml	android:label="ServeNow"
iOS name (Customer)	customer_app/ios/Runner/Info.plist	CFBundleDisplayName key
Android name (Provider)	provider_app/android/app/src/main/AndroidManifest.xml	android:label="ServeNow Provider"
Android name (Handyman)	handyman_app/android/app/src/main/AndroidManifest.xml	android:label="ServeNow Pro"
Splash screen titles	splash_screen.dart in all 3 mobile apps	Search "ServeNow" and replace
Admin web title	admin_web/lib/main.dart	title: "ServeNow Admin" in GetMaterialApp
Admin web sidebar	admin_web/lib/widgets/menu_widget.dart	ServeNow text in sidebar header

5.2 Change Package Names

```
# For each mobile app (customer_app, provider_app, handyman_app):
1. android/app/build.gradle → change applicationId
2. Rename Kotlin folder: com/serveNow/appname/ → com/yourbrand/appname/
3. Update package declaration in MainActivity.kt
4. In Xcode: change Bundle Identifier for iOS
5. Update Firebase Console – add new package names
6. Download fresh google-services.json and GoogleService-Info.plist

# For admin_web (web app – no package name needed):
# Just update the Firebase web app config via flutterfire configure
```

5.3 Update Colors and Theme

App	File to Edit	Key Colors
Customer App	lib/core/theme/app_colors.dart	primary (#4A6CF7), splashGradient, primaryGradient
Provider App	lib/core/theme/app_colors.dart in provider_app	Currently blue-purple — change primary values
Handyman App	lib/core/theme/app_colors.dart in handyman_app	Currently forest green (#2E7D32)
Admin Web	admin_web/lib/app/utils/app_colors.dart	primary (#6366F1), sidebarBg (#0F172A), background (#F8FAFC)

5.4 Replace App Icons and Logos

- Prepare 3 mobile app icons: 1024x1024 PNG, no transparency — one per app.
- Place each at: {app_folder}/assets/icons/app_icon.png
- In each mobile app folder run: `dart run flutter_launcher_icons`
- For admin_web: update admin_web/web/favicon.png and the logo in admin_web/lib/widgets/menu_widget.dart
- Replace splash screen logo assets in each mobile app.
- Rebuild all apps after icon changes.

6. Going Live Checklist

Firestore

- Replace all google-services.json and GoogleService-Info.plist with production versions
- Configure Firestore security rules to restrict access by auth and role
- Configure Firebase Storage security rules
- Enable Firebase App Check for production security
- Upgrade to Firebase Blaze plan — required for Cloud Functions
- Verify Cloud Functions deployed to asia-south1 (not us-central1)
- Delete any old us-central1 functions to prevent duplicate triggers

API Keys

- Replace Razorpay sandbox keys (rzp_test_) with live keys (rzp_live_) in functions/index.js and razorpay_service.dart
- Restrict Google Maps API key to your app package names in Google Cloud Console
- Enable billing on Google Cloud Console for Maps APIs
- Never commit API keys to version control

Mobile Apps — Release Builds

- Set minSdkVersion to 21 in all three mobile apps
- Generate a release keystore for each app using keytool command
- Configure release signing in android/app/build.gradle for each app
- Build release bundle: flutter build appbundle --release
- Test release builds on real physical devices before submitting

Admin Web Panel

- Run: flutter build web --release --web-renderer canvaskit
- Run: firebase deploy --only hosting
- Verify admin panel loads at your-project.web.app
- Test login, dashboard, provider approvals, handyman approvals, and booking management
- No server setup needed — Firebase Hosting handles everything

Play Store Submission

- Create Google Play developer account at play.google.com/console
- Prepare screenshots for all 3 apps (min 2 phone screenshots each)
- Write short and full app descriptions for each app
- Prepare a privacy policy URL — required for apps collecting personal data
- Set content rating for each app
- Submit for review — initial review typically 2 to 7 business days

7. Support & License

What's Included

- YES** Complete source code for all 4 apps (customer, provider, handyman, admin web)
- YES** Firebase Cloud Functions source code (Node.js — auto-assignment, FCM, commissions)
- YES** This documentation PDF
- YES** Demo data seeding script (scripts/seed_all.js) for realistic test data
- YES** All assets — app icons, Poppins font files, placeholder images
- YES** firebase.json and firestore.indexes.json for easy deployment

What's NOT Included

- NO** Firebase project (buyer must create their own free Firebase account)
- NO** Google Maps API key (buyer must generate their own in Google Cloud Console)
- NO** Razorpay merchant account (buyer must sign up at razorpay.com)
- NO** Apple Developer account for iOS App Store distribution
- NO** Ongoing maintenance, bug fixes, or new feature development

■ **No PHP server or hosting costs — the admin panel runs entirely on Firebase Hosting (free tier). No VPS, no cPanel, no monthly server bills.**

License Terms

This source code is sold under a single-use commercial license. The buyer may use this code to build and deploy one live production application. The code may not be resold, redistributed, or sub-licensed to third parties. Modification for personal or client projects is fully permitted. The original author provides no warranty and accepts no liability for issues arising from use of this code in production environments.

Thank you for purchasing ServeNow.

This platform is built entirely with Flutter and Firebase — no PHP servers, no Laravel, no hosting costs for the admin panel. Clean architecture and well-structured code makes customization straightforward for any Flutter developer.